

Module - I

Introduction

Basic Terminology

Data: Raw facts stored in memory.

Data Structure: A method to organize and store data efficiently.

Instruction: A program statement.

Instruction Types:

- Declaration Instruction
- I/O Instruction
- Arithmetic Instruction
- Control Instruction.

Data Types & Classification

A data type defines the set of values a variable can hold.

- Primitive (Built-in): int, float, char, double, void.
- Derived (Built from Primitive): Arrays, Pointers, Functions.
- User-defined (Custom): struct, union, enum, typedef.

Measuring Performance of Algorithm

Algorithm analysis is an important part of computational complexity theory, which provides theoretical estimation for the required resources of an algorithm to solve a specific computational problem. It is the determination of the amount of time and space resources required to execute it.

Q. Why analysis is required?

- Predicting behaviour of an algorithm without implementing.
- Analysis is only approximation, there're many more factors.
- It helps us in determining the best algorithm to solve a programming problem.

Type of Algorithm Analysis

- Certain input takes less time for an algorithm to accomplish its task i.e. best case. [Ω]
- Certain input takes maximum time for an algorithm to accomplish its task i.e. worst case. [O]
- All the other cases in which algorithm takes time for random/typical input is average case. [Θ]

Time Complexity

- It is measure of rate of change in time with respect to change in input size.
- Instead of measuring runtime in seconds, which varies based on hardware or processor speed, it counts the number of basic operations performed by the algorithm.
- It provides a standardized way to compare two different algorithms solving the same problem to see which is more efficient.
- It helps to predict how an algorithm will perform when processing massive amounts of data.
- There are various asymptotic notation to represent time complexity of an algorithm.

Asymptotic Notation

It is used to analyze performance of algorithm in term of input size. We do not calculate the actual time taken by the algorithm to solve the problem. Rather, we're interested in how the time taken by increasing with input size.

Big O Notation:

It provides upper bound on the growth rate of time and also ~~provides~~ represents the worst case.

Omega Notation:

It provides lower bound on the growth rate of time and represents the best case.

Theta Notation:

It provides both upper and lower bound on the growth rate of time and represents the average case.

(1)

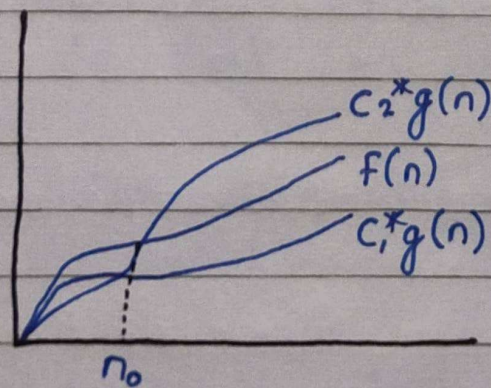


Fig.: ~~Big~~ Theta Notation

$$c_1 * g(n) \leq f(n) \leq c_2 * g(n)$$

c_1 & c_2 are the constant for all $n \geq n_0$.

$$f(n) = an^3 + bn + c$$

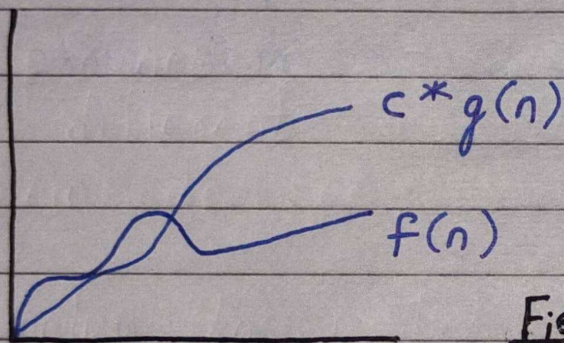
$$g(n) = 3n^3 + 2n + 5$$

$$g(n) = n^3$$

$$f(n) = \Theta(g(n^3))$$

$$\Theta(g(n^3)) = \Theta(n^3).$$

(2)



$$O(g(n)) = f(n)$$

$$0 \leq f(n) \leq g(n) \quad \forall n \geq n_2$$

Fig.: Big O Notation

$$f(n) = 2n^2 + 3n + 4$$

$$3f(n) > n^2$$

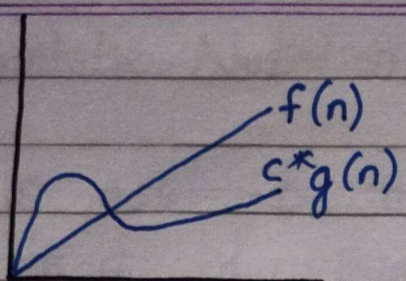
$$3n^2 > 2n^2 + 3n + 4$$

$$g(n) = 2n^2 + 3n + 4$$

$$O(g(n)) = O(n^2)$$

$$O(n^2).$$

(3)



$$\Omega(g(n)) = f(n)$$

$$c * g(n) \leq f(n) \quad \forall \quad n \geq n_0$$

Fig.: Omega Notation

$$f(n) = 2n^2 + 3n + 5$$

$$g(n) = n^2$$

$$\Omega(g(n)) = \Omega(n^2)$$

$$1 * n^2 < f(n)$$

$$1 * n^2 < 2n^2 + 3n + 5$$

$$\Omega(n^2).$$

Calculating $\text{Big } O$ for Factorial

```
f = 1  $\longrightarrow$   $c_1$   
if (n == 0)  $\longrightarrow$   $c_2$   
return 1
```

$$f(n) = c_1 + c_2 + n \cdot c_3 + c_4$$
$$\Rightarrow f(n) = n \cdot c_3 + c$$
$$[c = c_1 + c_2 + c_4]$$

```
for (i = 1 to n)  
  f = i * f  $\longrightarrow$   $c_3$   
return f  $\longrightarrow$   $c_4$ 
```

$$g(n) = n$$
$$f(n) = O(g(n)) = O(n).$$

$\text{Big } O$ of Insertion Sort Algorithm

```
for (i = 1 to n)  
  temp = A[i]  $\longrightarrow$   $c_1$ 
```

```
    for (j = i - 1 to 0)  
      if (A[j] > temp)  
        A[j+1] = A[j]  $\longrightarrow$   $i \cdot c_2$   
      else  
        break
```

```
  A[j+1] = temp  $\longrightarrow$   $c_3$ 
```

Cost for step $i = c_1 + i \cdot c_2 + c_3 = i \cdot c_2 + c$ [$c = c_1 + c_3$]

$$f(n) = \sum_{i=1}^n (i \cdot c_2 + c) = n^2 c_2 + n \cdot c_1$$

$$g(n) = n^2$$

$$f(n) = O(g(n)) = O(n^2).$$

Basic Formate Specifier

<u>Data Type</u>	<u>Formate Specifier</u>
int	→ %d or %i
float	→ %f
char	→ %c
double	→ %lf
String	→ %s

Program 1-1:

```
#include <stdio.h>
```

```
int  
void main()  
{
```

```
    char a,b,c;
```

```
    a = 65;
```

```
    b = 66;
```

```
    c = 67;
```

```
    printf("%c", a); // A
```

```
    printf("%c", b); // B
```

```
    printf("%c", c); // C
```

```
    return 0;
```

```
}
```

Program-2:

```
#include <stdio.h>
```

```
int main()  
{
```

```
    int arr[] = {10, 20, 30, 40, 50};
```

```
    int size = sizeof(arr);
```

```
    int length = sizeof(arr) / sizeof(int);
```

```
    printf("Size of array in bytes: %d", size); // 20  
    printf("\n Length of array: %d", length); // 5
```

```
    return 0;
```

```
}
```

Program-3:

```
#include <stdio.h>
```

```
int main() {
```

```
    float num = 3.5;
```

```
    printf("%f", num); // 3.5
```

```
    printf("\n %f", num); // 3.5
```

```
    printf("\n %.3f", num); // 3.500
```

```
    return 0;
```

```
}
```

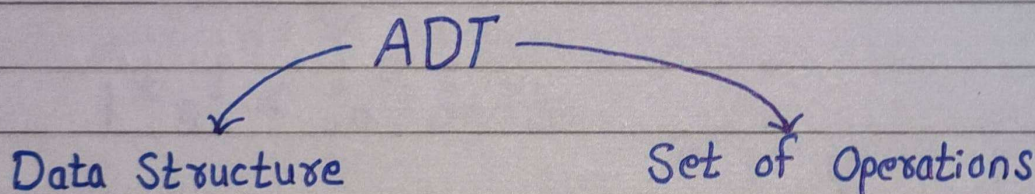
Data Structure

Data Structure is a particular way of storing and organising data in a computer so that it can be used efficiently.

Classification of Data Structure:

1. Linear Data Structure (Array, Linked List, Stack & Queue).
2. Non-linear Data Structure (Tree & Graph).

Abstract Data Type (ADT)



- All primitive data types support basic operation - like addition, subtraction, etc.
- The system is providing the implementation for the primitive data type.
- For non-primitive data type we also need to define operation.
- Combination of data structure and their operation are known as ADT.